

Rhylthyme: A Declarative Language and Agent-Native Runtime for Human-Executed, Resource-Constrained Schedules

Jeremy Leipzig
leipzig@gmail.com

September 2026

Abstract

Scheduling software tends to fall into three kinds: tools that compute a plan and hand it to a person as a static artifact; engines that execute a dependency graph on machines with no person in the loop; and checklists with timers that walk a person through steps without any model of shared resources or parallel work. Rhylthyme (“real time”) is built for what falls between them: capacity-constrained, multi-track schedules whose executor is a human working against a clock, with steps the executor starts and ends. We describe (i) a compact JSON language of parallel tracks, timed steps, dependency triggers, environments and resource constraints, annotated with workflow and time ontology terms; (ii) a validator, an auto-planner with several strategies, and two runtimes (a terminal runner and a web player with timeline, itinerary and dependency-graph views); and (iii) a Model Context Protocol server that exposes validation, timing analysis, catalog search, import and publication as annotated tools, so that a language-model agent can author and deliver a schedule as tool calls rather than prose. We position the system as a natural extension of traditional project scheduling techniques, and compare it with air-traffic flow management, temporal constraint networks and robot task scheduling, cooking-step and laboratory schedulers, lab-automation software, and workflow engines, arguing that its contribution is a narrow one: capacity constraints and cross-track dependencies inside a followable, human-executed timeline, one schema used across domains, and an agent interface whose validator refuses malformed programs before they are published.

1 Introduction

A great deal of practical scheduling has a person, not a machine, at the end of it. Abbott, Bogart and Walkingshaw call the artifacts involved “programs for people”: recipes, protocols and run sheets are written by one person to be executed by another, present an idealized linear process, and rely on conventions for widening or narrowing what a step means [1]. A cook coordinating four dishes so that they finish together, a technician running a protocol whose incubations overlap, a stage manager calling cues, and a trainer running supersets all face the same structure: several sequential lines of work that run in parallel, dependencies between them, durations that are sometimes fixed and sometimes open-ended, and a handful of resources (one oven, two thermocyclers, one PA system) that cannot be used by more than a bounded number of steps at once.

The tooling for this structure is split three ways. The first family comes from project management: the Gantt chart, a bar chart with one row per activity and time on the horizontal axis; the critical path method (CPM), which links activities by precedence and finds the chain that determines the finish date; the Program Evaluation and Review Technique (PERT), which does the same with three duration estimates per activity (optimistic, most likely, pessimistic) so that the finish date carries an uncertainty; and resource-constrained project scheduling, which adds limited resources that activities compete for. All of these produce a plan offline and leave execution to people and paper [8, 18, 23, 44]. Workflow engines and laboratory orchestrators execute dependency graphs, but their executors are compute nodes, robots or instruments, and their objective is reproducible output rather than a followable timeline [22, 39]. And a third family does put a person in the loop: protocol platforms with a run mode that steps through a checklist with a timer [38], guided-cooking timers, and show-control software that fires cues. What that family lacks is the first family’s model: parallel tracks, dependencies across them, and capacity limits on shared resources. Domain-specific schedulers for kitchens and laboratories have the model but stop at an optimized procedure [24, 28, 46].

Rhylthyme is a small system built for the intersection. Its contributions are:

1. **A language.** A JSON program model of parallel *tracks* of sequential *steps*, with a fixed vocabulary of start triggers, three kinds of duration, buffers, choice branches, replicates with staggered starts, and a separate *environment* document that declares resource capacities and the people available to attend them. The schema annotates its constructs with workflow and time ontology terms, so that each has a stated meaning in a standard temporal model.
2. **A runtime.** A validator that catches structural and scheduling errors, an auto-planner with named strategies (finish together, minimize length, provide consistent activity coverage, fit to a target time) and over-utilization detection, a curses runner with manual and indefinite steps, actor accounting and abort handling, and a web application whose timeline, itinerary and dependency-graph views play the schedule.
3. **An agent interface.** A Model Context Protocol (MCP) server that exposes `validate_program`, `analyze_schedule`, `visualize_schedule`, import from external recipe and protocol sources, catalog search and account tools, with tool annotations, structured output, resources and a prompt. One implementation serves five endpoints (generic, kitchen, laboratory, events, gym) from one schema.

2 Related work

2.1 Gantt charts, CPM and PERT

The Gantt chart is a century old and, as Wilson notes, its origins are murkier than its ubiquity suggests [44]. It remains the dominant visual grammar for “which activity runs when on which row,” and Rhythyme’s primary view is a Gantt chart whose rows are tracks. CPM [18] and PERT [23], both published in 1959, added precedence networks and earliest and latest start times; PERT, built for the US Navy’s Polaris programme, assumed each activity’s duration follows a beta distribution summarized by its three estimates. Rhythyme’s variable durations (`minSeconds`, `defaultSeconds`, `maxSeconds`) are a three-point estimate without PERT’s distributional assumptions, and its `analyze_schedule` tool reports the dependency chain that determines the makespan, a critical-chain heuristic rather than CPM’s float-based criticality (Section 5).

2.2 Resource-constrained project scheduling

The resource-constrained project scheduling problem (RCPSP) generalizes CPM with renewable resources of bounded capacity; Brucker et al. give the standard notation and classification [8], Kolisch and Padman survey exact and heuristic methods [19], and Pinedo covers the wider machine-scheduling family [33]. In those terms a Rhythyme track is a resource of capacity one (one pair of hands at one station: steps in a track may not overlap, and the validator rejects programs where they do) and each resource constraint is a renewable resource with capacity `maxConcurrent`. The difference is in what is done with the model: RCPSP solvers minimize makespan or net present value; Rhythyme checks feasibility, surfaces conflict windows, and applies staggering heuristics, leaving the final timing to the person running it.

That division of labour has its own tradition. Mixed-initiative scheduling systems of the 1990s paired a generative scheduler and constraint checker with a manual scheduler and critic so that a person and a program could revise one schedule together [16]; interactive decision-support systems for the resource-constrained scheduling problem let the decision maker change job priorities and priority rules and re-run the heuristic [29]; and the rescheduling literature argued early that practical scheduling is a reactive process in which circumstances continually force revision, so reactivity should be a design centre rather than an afterthought [17], with interactive visualization proposed as the medium through which machine reasoning and user expertise collaborate during rescheduling [25]. Rhythyme inherits the manual-editing half of that tradition (the web editor and the auto-planner strategies) and the visualization half (Section 6), but not the reactive half: it propagates delays along dependency chains at run time and does nothing more (Section 5). Timing also matters on the human side of the loop: Yu et al. show experimentally that the rhythm at which a mixed-initiative system takes and yields the initiative affects users’ sense of agency, performance and stress [45], which bears directly on how a runtime should fire audio cues and present manual gates.

2.3 Real-time flow management in air traffic control

Air-traffic flow management is the archetype of scheduling that is both resource-constrained and continuously replanned. Bertsimas and Stock Patterson formulate it as an integer program over airport and sector capacities and show it is NP-hard [7]; Ball et al. give a stochastic ground-holding model designed to integrate with Collaborative Decision Making (CDM) [5], the FAA/airline information-sharing regime of the 1990s [42]. Two ideas transfer, at a very different scale. Capacity is a cap on concurrent occupancy of a shared resource over a time window, which is Rhyllthyme’s `maxConcurrent`; and CDM’s premise that the plan is a shared artifact updated as participants act is the premise of Rhyllthyme’s shareable timeline, though Rhyllthyme has no optimization layer and does not yet merge progress across participants.

2.4 Temporal reasoning and robot task scheduling

Rhyllthyme’s triggers are temporal constraints between intervals. Allen’s interval algebra [2] names the qualitative relations and OWL-Time gives them a web vocabulary [15, 41]; the Rhyllthyme schema uses that vocabulary to annotate its constructs (Section 4). Dechter, Meiri and Pearl’s temporal constraint networks [10] add metric bounds; a Rhyllthyme program restricted to fixed durations, non-negative offsets and conjunctive triggers is a simple temporal network whose consistency the validator checks by resolving all start times. Temporal networks with uncertainty [27, 40] model durations the executor does not control; Rhyllthyme’s variable and indefinite steps are the other case, durations the executor *does* control by ending the step, and we return to the distinction in Section 9.

Robotics contributes the allocation side. Gerkey and Mataric’s taxonomy of multi-robot task allocation [12] and its extension to interrelated utilities and constraints by Korsah et al. [20] classify who does what; Gombolay, Wilcox and Shah’s Tercio schedules human–robot teams under tight temporospatial constraints with a satisficing sequencer inspired by real-time processor scheduling [13]. Rhyllthyme’s environments express the same allocation constraints declaratively (actor types with counts, per-task attention requirements and qualifications), but the runtime only accounts for them; it does not solve an assignment problem. Behavior trees [9] are the reactive-execution counterpart: like Rhyllthyme’s runner they execute a declarative structure tick by tick and react to manual and abort events, but they are control-flow trees rather than timed parallel tracks.

2.5 Domain schedulers: kitchens and laboratories

Cooking several dishes at once has been treated as a scheduling problem for decades. Guley and Stinson solved food-service production scheduling by branch and bound in 1984 [14]. Matsushima and Funabiki decompose recipes into six cooking-step types, schedule them under cook and utensil constraints, and present the result through a guidance interface [24]; Nakabe et al. build a task graph per recipe and present an optimal parallel procedure [28]. These are the closest prior systems to Rhyllthyme’s kitchen vertical. They differ in that each computes a procedure and presents it once, whereas Rhyllthyme keeps the resource model live during execution, with a clock, cues, manual steps, and the ability to save, share and re-run.

On the laboratory side, workflows that mix wet-lab and digital tasks were being reasoned about, for cost, quality and ordering, before the current wave of automation [21]. Self-driving laboratories (SDLs) have since made scheduling a first-class concern. Tom et al. survey the field [39]; the HELAO framework orchestrates distributed instruments running parallel, asynchronous experiments with device sharing and provenance [34]; Zhou et al. formulate multi-task SDL scheduling with operation precedence, station allocation and time-synchronization constraints under which one operation must begin the instant its predecessor ends [46]; the Artificial platform orchestrates whole-lab workflows, including manual steps performed by scientists [11]. Two recent lines of work sit closest to Rhyllthyme’s import and authoring paths: Shi et al. translate human-readable protocols into machine-interpretable protocol dependence graphs while preserving causality and consistency [35], and Angelopoulos et al. give an orchestration system an LLM agent that creates and monitors protocols from natural language, with automated validation and error correction and the ability to alternate between agent-assisted and manual construction [3]. Both target robotic execution; the same moves (structured translation, validate-and-correct authoring) are what Rhyllthyme applies to protocols a person will run. Beneath the schedulers sits a layered software landscape: device-level standards and drivers (SiLA 2’s typed feature definitions over gRPC [36], the Openrons Python Protocol API [31], PyLabRobot’s hardware-agnostic liquid-handling interface [43]); a machine-readable protocol layer (Autoprotocol’s JSON protocol description [37]); and a human-readable protocol layer (protocols.io, whose run mode already gives a scientist a checklist with a timer [38], and electronic notebooks

such as Benchling). Rhyllhyme sits at the human end of this stack but imports from both ends: it converts protocols.io and Benchling protocols *and* Opentrons protocol scripts into timed, resource-aware programs a bench scientist runs by hand, and its `afterStep` trigger with zero offset is the human-executed analogue of the SDL synchronization constraint.

2.6 Workflow engines and process notations

Configuration-based bioinformatics pipeline frameworks are the closest *architectural* relatives: a declarative description of steps and dependencies, a validator, an execution engine and a set of importers. We previously surveyed these frameworks along three axes: implicit versus explicit syntax, configuration/convention/class design paradigm, and command-line versus workbench interface [22]. On those axes Rhyllhyme is an explicit-syntax, configuration-paradigm system offered through both a command line and a web workbench. What separates it from the frameworks is not architecture but executor and objective: pipeline engines run on machines and optimize for reproducible output across compute environments, and some now expose their platforms to agents through vendor MCP servers; Rhyllhyme runs on a person and optimizes for a timeline that can be followed. The same contrast holds against the newest member of the family, an MCP-native workflow engine in which an agent reasons once to produce a declarative JSON blueprint of tool calls, with loops, parallel branches and data piping, that is then executed without further reasoning [32]: a Rhyllhyme program is also a declarative JSON document produced by an agent, but its steps are executed by a person at the stove or the bench, not by the engine. The nearest analogue in the human-process world is BPMN 2.0 [30], whose execution semantics include user tasks assigned to people and timer events; BPMN models control flow, roles and messages, and leaves capacity limits on shared resources and the wall-clock playback of a schedule to the engine that hosts it.

3 The Rhyllhyme program model

A Rhyllhyme program is a JSON document validated against a JSON Schema (version 0.2.0-alpha at the time of writing). Figure 1 shows a five-track Thanksgiving dinner planned around one oven, rendered by the bundled timeline renderer; Listing 1 shows three of its steps. Table 1 defines the concepts the language shares with every schedule of this kind, with one example from each vertical.

Listing 1: Three steps from Figure 1: an indefinite roast that occupies the oven until the cook ends it; a step that starts 45 minutes before the roast is due to end; and a manual gate that the host fires when guests sit down.

```
{ "stepId": "turkey-roast", "name": "Roast until 74°C", "task": "oven",
  "duration": { "type": "indefinite", "defaultSeconds": 9900 },
  "startTrigger": { "type": "afterStep", "stepId": "turkey-prep" } },
{ "stepId": "potatoes-peel", "name": "Peel and cut", "task": "prep",
  "duration": { "type": "fixed", "seconds": 900 },
  "startTrigger": { "type": "afterStep", "stepId": "turkey-roast",
    "offsetSeconds": -2700 } },
{ "stepId": "guests-seated", "name": "Guests seated", "task": "host",
  "duration": { "type": "fixed", "seconds": 300 },
  "startTrigger": { "type": "manual", "triggerName": "guests-seated" } }
```

Tracks, steps and triggers. A program has a `programId`, `name`, optional `environmentType`, optional `actors` or an `environment` reference, an array of `tracks`, optional `resourceConstraints` and free-form `metadata` (ingredients, serves, source attribution, thumbnail). A track is a named sequence of steps that must not overlap in time; the array order of steps is not itself enforced, only non-overlap. Step identifiers are unique across the whole program so that any step may reference any other. A start trigger is one of `programStart`, `programStartOffset`, `afterStep` (anchored to the referenced step’s end, or its start with `event: start`, plus a signed `offsetSeconds`), `afterStepWithBuffer`, `manual` and `onAbort`, or a compound `{logic: all|any, triggers: [...]}` that waits for all, or the first, of several. Offsets and durations accept a number of seconds or a unit string ("5m", "1h30m"). A trigger may carry a `choiceId`; a step with a `choice` block presents options at run time, and downstream steps gated on a `choiceId` run only for that option. A negative `afterStep` offset ("start 20 minutes before the roast is done") is accepted only against an indefinite step; planners place it against the referenced step’s `defaultSeconds`, and the runtimes fire it from that forecast rather than from the actual end, a limitation discussed in Section 9.

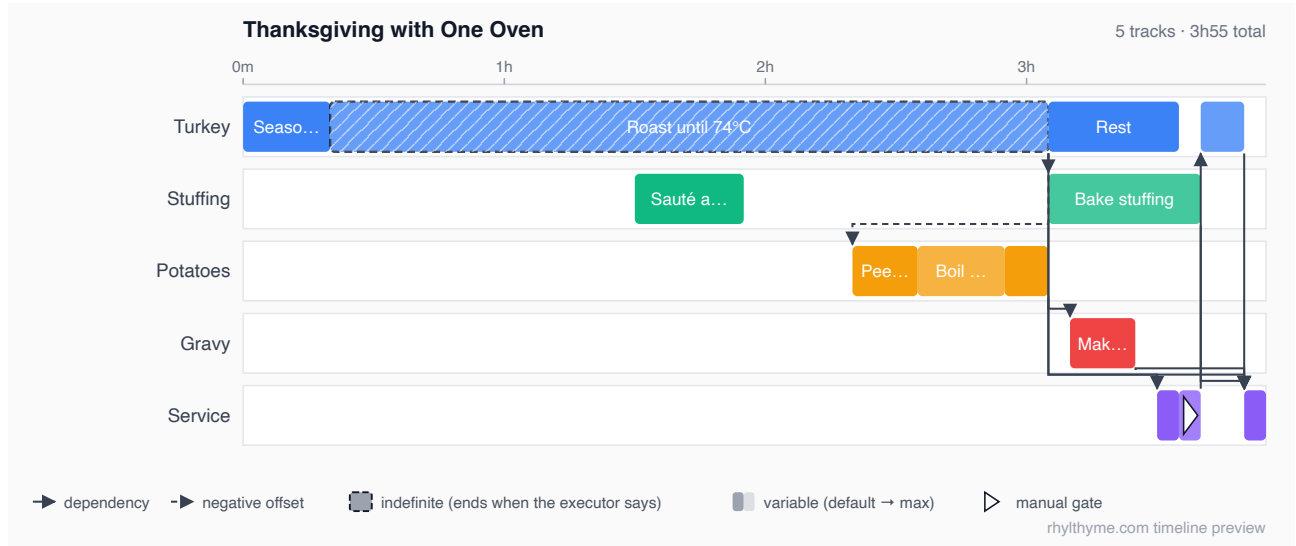


Figure 1: *Thanksgiving with One Oven* (five tracks, thirteen steps, 3 h 55 at planned durations) as rendered by the open-source timeline renderer that also produces the MCP server’s image previews. Rows are tracks; bars are steps positioned by resolved start time. The roast is *indefinite* (hatched, dashed border): it ends when the cook says so, and everything downstream is planned against its default duration. Arrows are cross-track dependencies; the dashed arrow is a negative offset (peel the potatoes 45 min before the roast is due out). The flag on “Guests seated” marks a *manual* gate that the host fires; “Carve” waits for it, and “Serve” waits for all four dishes. The oven is a capacity-one resource, so the stuffing bakes only after the turkey comes out. The web application draws its own D3 timeline from the same JSON.

Durations. A *fixed* duration has `seconds`. A *variable* duration has `minSeconds` and `maxSeconds`, an optional `defaultSeconds` used for planning, and may be ended early by the executor. An *indefinite* duration runs until the executor ends it; `defaultSeconds` gives planners something to draw. In every case the person running the program decides when a non-fixed step ends; the language has no construct for a duration that is uncertain *and* outside the executor’s control.

Environments. An environment is a separate JSON document, validated against its own schema, that describes a workplace once so that many programs can share it. It has an `environmentId`, a `type` from a controlled list (37 values, from `kitchen` and `laboratory` to `theater`, `gym` and `datacenter`), `actorTypes` (named categories of people with a `count`), and `resourceConstraints`. Each constraint names a task, its `maxConcurrent` capacity, the fraction of a person it occupies (`actorsRequired`: an oven needs 0.5 of a cook, a microwave 0.25, a stove burner 1.0), and which actor types are `qualifiedActorTypes` for it. A program references an environment by id; the loader falls back to the default environment for the program’s `environmentType` when none is named, and inline `resourceConstraints` on the program take precedence. The repository ships ten environments (home and commercial kitchens, academic, biotech and pharma laboratories, a bakery, an airport, a software team), and `create_environment` on the MCP server produces new ones. A program that names neither an environment nor `actors` must declare a resource constraint for every task it uses. Environments are what let the same protocol be scheduled differently in a lab with one thermocycler and a lab with four; the attention and qualification fields are honoured by the command-line runner, while the web and MCP engines use only capacities and the program-level actor count.

Replicates and staggered starts. Repetition has a shorthand so that authors need not copy tracks. A track may declare `batch_size` and `stagger`: `"batch_size": 3, "stagger": "1m30s"` expands into three copies of the track whose starts are 90 seconds apart, and `templateId` with `templateParameters` lets those copies instantiate a shared `trackTemplates` entry with different values (three landings on one runway with different flight numbers, in the airport example). A track or step may instead declare `replicates` with a `count`, a `mode` of `parallel`, `stagger` or `serial`, and a `delay`; step-level parallel replicates fan out into sub-tracks that rejoin,

serial replicates chain within the track, and staggered replicates delay each start by the given amount. Expansion happens before validation in the Python toolchain, so its validator and runner only ever see plain tracks and steps; the JavaScript engine behind the MCP server validates the unexpanded form, which is the one known point where the two validators can disagree (Section 5).

4 Ontologies and vocabularies

The program and environment schemas carry `$comment` annotations written in the vocabulary of the W3C Time Ontology in OWL [15, 41], itself grounded in Allen’s interval relations [2]. A step is a `time:ProperInterval` with a `time:hasBeginning` and `time:hasEnd`; a fixed duration is a `time:Duration` with `time:numericDuration` in `time:unitSecond`; `afterStep` with zero offset is `time:intervalMetBy` and with a positive offset `time:intervalAfter`; a pre-buffer `time:intervalMeets` its step and a post-buffer is `time:intervalMetBy` it; `programStartOffset` places a `time:Instant` relative to program start; compound triggers are conjunctions or disjunctions of these relations. The annotations are documentation, not an RDF export: they fix the intended meaning of each construct for implementers of the validator, the two runtimes and the renderer, and they make the mapping to the temporal-reasoning literature explicit rather than folkloric. Generating OWL-Time individuals from a program is a small transformation we have not built. A more ambitious use of the same annotations would compile them into the tool interface itself, so that an agent could only construct programs that satisfy the ontology’s constraints; Zhou et al. demonstrate that ontology-to-tools compilation for LLM agents on a knowledge-graph platform [47]. Rhythyme enforces its constraints after generation, through the validator, rather than during it.

Three smaller controlled vocabularies do practical work. The environment `type` list names the kinds of workplace the system knows how to default. An instrument catalog groups laboratory and kitchen equipment into categories (centrifuges, thermal cyclers, plate readers, liquid handling, cold storage, ovens, ranges, mixers, proofers, and so on) with default capacities and attention requirements, so that an environment can be assembled by picking instruments. And the public recipe catalog is faceted by a fixed taxonomy of categories, cuisines, diets, time buckets, main ingredients, methods and tags, which the search API validates strictly so that a mistyped facet fails loudly rather than silently returning nothing.

5 Runtime and tooling

Validation. There are two validators. The Python one, used by the command-line tools, runs JSON Schema validation and then logic checks: duplicate or missing identifiers, references to non-existent steps, choice references to options that do not exist, tasks without a resource constraint, and, after resolving every start time, steps that overlap within a track. The JavaScript one, used by the MCP server, performs the same logic checks without the schema pass, adds dependency-cycle and unparseable-duration detection, and reports each finding with a stable code, a message and a *fix* hint (“chain `s2` with `{type: afterStep, stepId: s1}` or move it to its own track”), which matters when the author is a language model that will act on the message verbatim. The top-level example corpus holds 38 JSON programs (15 kitchen, 6 laboratory, 4 event, 3 fitness, 2 bakery, 2 airport, 1 manufacturing, 5 general or test). The JavaScript validator rejects four of them, three for within-track overlaps and one for undeclared resources; the Python validator rejects three, the difference being one program whose overlap disappears once replicates are expanded. The remaining semantic differences between the two are listed in Section 9.

Common optimizations. The auto-planner, available from the web editor and as a Python module, works from earliest start times under the dependency graph, checks the result for resource over-utilization (intervals where more steps occupy a task than its capacity allows), and offers four named strategies that correspond to what people actually ask a schedule to do: *synchronized finish* delays shorter tracks so that every track ends at the same moment (dinner on the table together); *minimize length* packs steps as early as dependencies allow; *provide consistent activity coverage* sequences steps one after another, alternating tracks, so that a single executor always has something to do and no long idle gaps open up; and *fit to time* compresses flexible steps and omits low-priority ones until the program meets a target duration, using each step’s `priority` and `flex` settings (a flexible step may *fill* the gap to its successor or stretch to its `maxDuration`). All four are heuristics over earliest-start times and dependency order, not solvers; they report over-utilization rather than resolving it, and

their job is to hand the person a feasible schedule and to say where the pressure is. (An older planner in the core library predates the current schema and is not described here.)

Analysis. `analyze_schedule` resolves all start times and computes the makespan; the chain of predecessors that determines it (walking back from the last-ending step through the predecessor whose end is closest to each start); resource-conflict windows by a sweep line over each task’s occupancy; peak concurrency against declared actors; per-track slack before the program’s end; and, given a `finishAt` or `startAt` instant, the wall-clock start of every step. “When do I put the potatoes in so that everything is ready at six” is a single call.

Execution. Two runtimes play a program. The command-line runner is a curses interface that advances a clock, starts steps as their triggers fire, waits on manual triggers, lets the executor end variable and indefinite steps, accounts for actor capacity from the environment, and handles abort with `onAbort` triggers. Because `afterStep` triggers fire on actual completion, a step that runs long delays everything chained after it, while steps anchored to program start by offset stay on the clock; there is no replanning beyond that propagation. The web application is a player: it advances a simulated clock on a browser interval timer at a chosen speed, starts and highlights steps as their resolved times arrive, and lets the executor end non-fixed steps; it is not re-anchored to wall-clock time while running, so a phone that sleeps mid-roast pauses the schedule rather than catching up. It renders the program in three principal views and several auxiliary ones (Section 6). Both runtimes consume the same JSON, and both are what the MCP server links to.

Importers. A registry of importers, plus a separately authenticated Benchling connector, converts external sources into programs: Spoonacular and TheMealDB recipes, Cooklang files, protocols.io and Benchling protocols, and Opentrons Protocol API scripts. Importers infer tracks and durations from source structure, a much weaker form of the protocol-to-dependence-graph translation that Shi et al. pursue for robotic labs [35], and that bounds their quality: a recipe or protocol source typically yields one or two tracks, and the multi-track programs in the corpus are authored by hand or by an agent. The output is validated like any other program. The Opentrons importer runs the protocol script in the Opentrons simulator to recover its command sequence, which executes untrusted code and is sandboxed only by process isolation on the server; we return to this in Section 9.

6 Views in the web application

Timeline. The timeline is a Gantt chart with one row per track and one bar per step, positioned by resolved start time and coloured by track (Figure 1 is the static form). It carries the execution controls: start, pause and stop, a speed multiplier for rehearsal, a zoom slider, and a time-format selector that switches the axis between elapsed minutes and time of day from a chosen start. While running, a cursor sweeps the chart, active steps are highlighted, audio cues fire at step boundaries, and variable and indefinite steps expose a control to end them. On the kitchen vertical an ingredient checklist sits beside it.

Itinerary. The itinerary is the same schedule sorted by clock time regardless of track: a chronological list of “at 17:12 start the potatoes” entries with each step’s track, duration and description. It is the view a person reads when they want to know what to do next rather than how the tracks relate, and it is the text form the MCP server returns in its previews.

Dependency graph. The DAG view draws steps as nodes and triggers as edges, with buffered edges styled distinctly, so that cross-track dependencies and choice branches, which the timeline can only show by position, are explicit. It is the view used when editing a program’s structure.

Others. A resources view plots each task’s concurrent usage against its capacity over time, which is where over-utilization is seen; a radial view wraps the timeline around a clock face; a kanban view groups steps by state during execution; a fold view collapses the program into the sequence of merge points; an editor view exposes drag-to-reschedule and the auto-planner strategies of Section 5; and a video view plays step media where a program provides it. The web application also hosts a public catalog of programs, per-user libraries, and per-vertical hosts that differ only in defaults and copy.

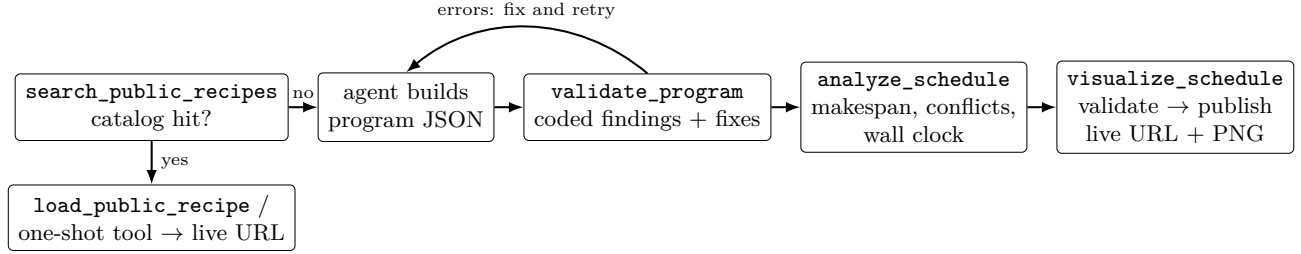


Figure 2: The workflow the server’s `instructions` describe to the host at initialization. Catalog content is returned with its live URL directly; new content is validated before it can be published.

7 The MCP server: agent-native authoring

The Model Context Protocol [4] standardizes how a language-model host discovers and calls tools, reads resources and invokes prompts; the server described here uses the Streamable HTTP transport and tool annotations introduced in the 2025-03-26 revision and the structured tool output of 2025-06-18 [26]. It is deployed as a stateless endpoint and served at five paths, `/mcp`, `/kitchen/mcp`, `/lab/mcp`, `/events/mcp` and `/gym/mcp`, from one implementation: each vertical differs from the generic endpoint only in default catalog filter, viewer host, copy, and two added convenience tools (a one-shot “find the best match and open it” verb such as `cook_recipe` or `run_protocol`, and a random pick). Figure 2 shows the intended interaction.

Tools. Each of the thirteen tools is registered with a title and MCP annotations (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`); four (validation, analysis, search and publication) also carry an output schema and return `structuredContent`, so that hosts which honour annotations can auto-approve read-only calls and agent frameworks can consume results without parsing markdown. The annotations are self-declared hints, and a host is free to ignore them. `validate_program` and `analyze_schedule` are pure and read-only. `visualize_schedule` validates first and by default refuses an invalid program with the same coded findings (an explicit `allowInvalid` flag publishes a rough draft anyway), then creates a share record and returns a markdown preview (cover image, equipment, ingredients, an ASCII Gantt, the itinerary and a schedule check), an inline PNG of the timeline, and the live URL. `import_from_source` wraps the importer registry; `create_environment` produces an environment document; `search_public_recipes` and `load_public_recipe` read the public catalog; `login`, `list_my_programs`, `load_program` and `save_program` handle a user’s own library, with `save_program` also validating first. Failures set `isError` and say what to do next, including when a call needs a login token.

Resources and prompts. The server publishes the program JSON Schema, a one-page authoring guide (the rules the validator enforces, the trigger vocabulary, how to make tracks finish together, choice branching), and five complete example programs as MCP resources, and a `plan_schedule` prompt that takes a goal, an optional deadline and optional resource limits and walks the model through the workflow in Figure 2. Server-level `instructions` carry the workflow once per session rather than repeating it in every tool description.

Why this matters. In our experience the failure mode of an LLM asked to “plan Thanksgiving with one oven” is a wall of prose that is wrong about timing and impossible to follow at the stove. Giving the model a validator with fix hints, an analyzer that answers “when do I start,” and a publisher that refuses malformed input by default moves the burden of *structural* correctness (identifiers, dependencies, overlaps, declared resources) from the model’s text to the tool contract, and the deliverable from a chat message to a page on the cook’s phone. Whether the durations and orderings the model chooses are sensible is not something the validator can check. The pattern is not unique to Rhythyme: Benyamin et al.’s Planning Copilot exposes syntax checking, planner selection, plan validation and execution simulation to an LLM as MCP tools for the same reason [6], and Angelopoulos et al. pair natural-language protocol authoring with automated validation for a lab orchestrator [3]. What Rhythyme adds is the domain (a person’s timeline rather than a planner’s plan or a robot’s protocol) and the publication step. We report this as a design, not a measurement; an evaluation of agent-authored programs against the validator is future work.

8 Discussion: positioning

Table 2 summarizes the comparison. Rhyllhyme *borrow*s the Gantt row-per-track grammar, the RCPSP resource model, the idea of a critical chain, and the trigger vocabulary of temporal constraint networks. It *omits* what most of the compared systems are built around: it does not optimize an objective, does not reason about durations outside the executor’s control, does not solve actor assignment, and does not execute anything on a machine. What is *new* is narrower than a new model of human work: in the language and both runtimes the person is a capacity-one resource whose duration oracle is a button, and the human-specific constructs are the triggers that wait for that button, the attention fractions in environments, and the affordances of the runtimes. The claim is that no compared system combines capacity constraints and cross-track dependencies with a followable, human-executed timeline that has manual, executor-ended and abort steps; that the same schema is used unchanged across cooking, laboratory, event and training verticals, with domain content kept in metadata and environments rather than in the schema; and that validation, analysis and publication are exposed as typed, annotated tool calls rather than as instructions in a prompt. Table 2 compares traditions rather than individual systems, so its cells describe the mainstream of each column, and specific systems (a workflow platform’s MCP server, a lab orchestrator’s guided manual steps) already occupy some of the cells the Rhyllhyme column claims.

The nearest neighbours deserve a direct comparison. The cooking-step schedulers of Matsushima and Funabiki and of Nakabe et al. [24, 28] and the SDL scheduler of Zhou et al. [46] model the same objects (parallel procedures, shared stations, tight synchronization) and solve for an optimized procedure; Rhyllhyme would be a natural *target* for their output, since none produces a shareable runtime or an agent interface. Protocol run modes and guided-cooking timers are the other neighbour: they have the runtime but not the model, so a second dish or a second protocol cannot be placed against the first. The mixed-initiative scheduling workbenches [16, 29] had both a model and a human in the loop, but the human’s role was to edit the schedule, not to execute it. Workflow engines and lab orchestrators, finally, would be natural *consumers* of Rhyllhyme’s trigger vocabulary if the executor were a machine; the language is agnostic about that, but the runtimes are not.

9 Limitations and future work

The auto-planner is a heuristic with no optimality guarantees. Variable and indefinite steps are ended by the executor, so a program’s uncertainty is under the executor’s control; the case the temporal-networks literature treats, a step whose end is decided by a process (an incubation, a rise), is not represented, and adding it would make dynamic-controllability checking [27] the right tool for “start twenty minutes before the roast is done.” Today that construct is weaker than it reads: the negative offset is resolved against the indefinite step’s planned duration and the runtimes fire it from that forecast, so a roast that finishes early or late is not compensated. The two validators also differ: only the Python one applies the JSON Schema and expands replicates, only the JavaScript one detects cycles and unparseable durations, and the Python one does not yet accept unit strings such as “30s”; a cross-implementation test over the corpus, and a single specification both implement, are overdue. The web application’s clock is a simulated player clock, not a wall-clock dispatcher. Actor constraints are accounted for but not solved; a small assignment layer along the lines of Tercio [13] is the obvious next step for multi-person kitchens and labs. When several people follow one program, each runs their own copy of the clock; merging progress across participants, the CDM idea, is not implemented, and the reactive rescheduling that the scheduling literature has treated as central since the 1990s [17] is limited to delay propagation. The rhythm of cues and manual gates is fixed by the program; the evidence that interaction timing shapes users’ agency [45] suggests it should be a design parameter. The reported critical chain is a heuristic and not a float-based critical path, and the web application computes step timings with its own routine, so three timing engines (Python validator, renderer, web timeline) coexist without a test that they agree. Import quality is bounded by source structure, recipe text under-specifies durations, and the Opentrons importer executes fetched protocol code, which needs a proper sandbox before it should accept arbitrary URLs. The MCP server’s login is a pasted, short-lived bearer token that passes through the model’s context, outside the protocol’s OAuth framework; adopting it would remove that step and that exposure for consumer hosts. Four example programs fail the validator because of genuine within-track overlaps, which is evidence both that the checks are useful and that authoring remains error-prone. Finally, the language has no formal semantics beyond the schema, its OWL-Time annotations and the reference validators; writing one down would make the controllability and assignment work well-posed.

10 Conclusion

Rhylthyme is a small, opinionated answer to a common situation: a person, a clock, several things happening at once, and not enough ovens. It reuses well-understood ideas from project scheduling and temporal reasoning, packages them in a schema that survives being moved between kitchens, laboratories, stages and gyms, plays the result live with the resource model still in force, and, through MCP, lets a language model do the authoring against a tool contract that refuses by default to publish what will not work.

Availability. The specification, example programs and environments, and the command-line runner are public at <https://github.com/rhylthyme>; the web application, importers and remote MCP server implementation are being prepared for release under the same organization. The remote server is at <https://mcp.rhylthyme.com/mcp>. This report describes the repositories as of September 2026 and is released under CC BY 4.0.

References

- [1] Keeley Abbott, Christopher Bogart, and Eric Walkingshaw. Programs for people: What we can learn from lab protocols. In *2015 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 203–211, 2015. doi: 10.1109/VLHCC.2015.7357218.
- [2] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11): 832–843, 1983. doi: 10.1145/182.358434.
- [3] Angelos Angelopoulos, James F. Cahoon, and Ron Alterovitz. From prompts to protocols: An AI agent for laboratory automation. arXiv:2605.16552, 2026.
- [4] Anthropic. Introducing the Model Context Protocol. Announcement, 25 November 2024; specification revision 2024-11-05, 2024. URL <https://www.anthropic.com/news/model-context-protocol>. Accessed 2026-09-08.
- [5] Michael O. Ball, Robert Hoffman, Amedeo R. Odoni, and Ryan Rifkin. A stochastic integer program with dual network structure and its application to the ground-holding problem. *Operations Research*, 51(1): 167–171, 2003. doi: 10.1287/opre.51.1.167.12795.
- [6] Yarin Benyamin, Argaman Mordoch, Shahaf S. Shperberg, and Roni Stern. Toward PDDL planning copilot. arXiv:2509.12987, 2025.
- [7] Dimitris Bertsimas and Sarah Stock Patterson. The air traffic flow management problem with enroute capacities. *Operations Research*, 46(3):406–422, 1998. doi: 10.1287/opre.46.3.406.
- [8] Peter Brucker, Andreas Drexler, Rolf Möhring, Klaus Neumann, and Erwin Pesch. Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research*, 112(1):3–41, 1999. doi: 10.1016/S0377-2217(98)00204-5.
- [9] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018. doi: 10.1201/9780429489105.
- [10] Rina Dechter, Itay Meiri, and Judea Pearl. Temporal constraint networks. *Artificial Intelligence*, 49(1–3): 61–95, 1991. doi: 10.1016/0004-3702(91)90006-6.
- [11] Yao Fehlis, Paul Mandel, Charles Crain, Betty Liu, and David Fuller. Accelerating drug discovery with Artificial: a whole-lab orchestration and scheduling system for self-driving labs. arXiv:2504.00986, 2025.
- [12] Brian P. Gerkey and Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *International Journal of Robotics Research*, 23(9):939–954, 2004. doi: 10.1177/0278364904045564.
- [13] Matthew C. Gombolay, Ronald J. Wilcox, and Julie A. Shah. Fast scheduling of robot teams performing tasks with temporospatial constraints. *IEEE Transactions on Robotics*, 34(1):220–239, 2018. doi: 10.1109/TRO.2018.2795034.

- [14] H. M. Guley and J. P. Stinson. Scheduling and resource allocation in a food service system. *Journal of Operations Management*, 4(2):129–144, 1984. doi: 10.1016/0272-6963(84)90028-7.
- [15] Jerry R. Hobbs and Feng Pan. An ontology of time for the semantic web. *ACM Transactions on Asian Language Information Processing*, 3(1):66–85, 2004. doi: 10.1145/1017068.1017073.
- [16] Wen-Ling Hsu, Michael Prietula, Gary Thompson, and Peng Si Ow. A mixed-initiative scheduling workbench integrating AI, OR and HCI. *Decision Support Systems*, 9(3):245–257, 1993. doi: 10.1016/0167-9236(93)90056-9.
- [17] Gerry Kelleher. Scheduling and rescheduling. In *IEE Colloquium on Artificial Intelligence in Manufacturing*. IET, 1997. doi: 10.1049/ic:19970343.
- [18] James E. Kelley and Morgan R. Walker. Critical-path planning and scheduling. In *Proc. Eastern Joint IRE-AIEE-ACM Computer Conference*, pages 160–173, 1959. doi: 10.1145/1460299.1460318.
- [19] Rainer Kolisch and Rema Padman. An integrated survey of deterministic project scheduling. *Omega*, 29(3): 249–272, 2001. doi: 10.1016/S0305-0483(00)00046-3.
- [20] G. Ayorkor Korsah, Anthony Stentz, and M. Bernardine Dias. A comprehensive taxonomy for multi-robot task allocation. *International Journal of Robotics Research*, 32(12):1495–1512, 2013. doi: 10.1177/0278364913496484.
- [21] Zoé Lacroix, Christophe Legendre, and Serkan Tüzmen. Reasoning on scientific workflows. In *2009 IEEE Congress on Services (SERVICES-I)*, pages 306–313, 2009. doi: 10.1109/SERVICES-I.2009.73.
- [22] Jeremy Leipzig. A review of bioinformatic pipeline frameworks. *Briefings in Bioinformatics*, 18(3):530–536, 2017. doi: 10.1093/bib/bbw020.
- [23] D. G. Malcolm, J. H. Roseboom, C. E. Clark, and W. Fazar. Application of a technique for research and development program evaluation. *Operations Research*, 7(5):646–669, 1959. doi: 10.1287/opre.7.5.646.
- [24] Yukiko Matsushima and Nobuo Funabiki. A cooking-step scheduling algorithm with guidance system for homemade cooking. *IEICE Transactions on Information and Systems*, E98-D(8):1439–1448, 2015. doi: 10.1587/transinf.2015EDP7048.
- [25] George Melissargos and Pearl Pu. Employing interactivity and visualization to augment the process of machine-based rescheduling. In *Proceedings of the First International Workshop on Approximate Reasoning in Scheduling (ARS’97), International ICSC Symposium on Fuzzy Logic and Applications (ISFL’97)*, Zurich, 1997.
- [26] Model Context Protocol contributors. Model Context Protocol specification, revision 2025-06-18. Streamable HTTP transport and tool annotations (2025-03-26); structured tool output (2025-06-18), 2025. URL <https://modelcontextprotocol.io/specification/2025-06-18>. Accessed 2026-09-08.
- [27] Paul Morris, Nicola Muscettola, and Thierry Vidal. Dynamic control of plans with temporal uncertainty. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI-01)*, pages 494–502, Seattle, 2001.
- [28] Jin Nakabe, Teruhiro Mizumoto, Hirohiko Suwa, and Keiichi Yasumoto. Optimal cooking procedure presentation system for multiple recipes and investigating its effect. *Algorithms*, 14(2):67, 2021. doi: 10.3390/a14020067.
- [29] Mario Norbis and J. MacGregor Smith. An interactive decision support system for the resource constrained scheduling problem. *European Journal of Operational Research*, 94(1):54–65, 1996. doi: 10.1016/0377-2217(95)00187-5.
- [30] Object Management Group. Business Process Model and Notation (BPMN), version 2.0. Object Management Group, formal/2011-01-03, 2011. URL <https://www.omg.org/spec/BPMN/2.0/>. Accessed 2026-09-08.
- [31] Opentrons Labworks. Opentrons Python Protocol API, version 2. Documentation, 2026. URL <https://docs.opentrons.com/python-api/>. Accessed 2026-09-08.

- [32] Abhinav Singh Parmar. Separating intelligence from execution: A workflow engine for the Model Context Protocol. arXiv:2605.00827, 2026.
- [33] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, sixth edition, 2022. doi: 10.1007/978-3-031-05921-6.
- [34] Fuzhan Rahmanian, Jackson Flowers, Dan Guevarra, Matthias Richter, Maximilian Fichtner, Phillip Donnelly, John M. Gregoire, and Helge S. Stein. Enabling modular autonomous feedback-loops in materials science through hierarchical experimental laboratory automation and orchestration. *Advanced Materials Interfaces*, 9(8):2101987, 2022. doi: 10.1002/admi.202101987.
- [35] Yu-Zhe Shi, Fanxu Meng, Hao-fei Hou, Zhang-qian Bi, Qiao Xu, Lecheng Ruan, and Qining Wang. Expert-level protocol translation for self-driving labs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. doi: 10.48550/arXiv.2411.00444.
- [36] SiLA Consortium. SiLA 2 standard (v1.0). SiLA Consortium, 2019. URL <https://sila-standard.com/standards/>. Accessed 2026-09-08.
- [37] Strateos. Autoprotocol specification. Strateos (formerly Transcriptic), 2023. URL <https://autoprotocol.org/>. Accessed 2026-09-08.
- [38] Leonid Teytelman, Alexei Stoliartchouk, Lori Kindler, and Bonnie L. Hurwitz. Protocols.io: Virtual communities for protocol development and discussion. *PLoS Biology*, 14(8):e1002538, 2016. doi: 10.1371/journal.pbio.1002538.
- [39] Gary Tom, Stefan P. Schmid, Sterling G. Baird, Yang Cao, Kourosh Darvish, Han Hao, Stanley Lo, Sergio Pablo-García, Ella M. Rajaonson, Marta Skreta, Naruki Yoshikawa, Samantha Corapi, Gun Deniz Akkoc, Felix Strieth-Kalthoff, Martin Seifrid, and Alán Aspuru-Guzik. Self-driving laboratories for chemistry and materials science. *Chemical Reviews*, 124(16):9633–9732, 2024. doi: 10.1021/acs.chemrev.4c00055.
- [40] Thierry Vidal and Hélène Fargier. Handling contingency in temporal constraint networks: from consistency to controllabilities. *Journal of Experimental & Theoretical Artificial Intelligence*, 11(1):23–45, 1999. doi: 10.1080/095281399146607.
- [41] W3C. Time Ontology in OWL. W3C Candidate Recommendation, 26 March 2020, 2020. URL <https://www.w3.org/TR/2020/CR-owl-time-20200326>. Accessed 2026-09-08.
- [42] Michael Wambsganss. Collaborative decision making through dynamic information transfer. *Air Traffic Control Quarterly*, 4(2):109–125, 1996. doi: 10.2514/atcq.4.2.109.
- [43] Rick P. Wierenga, Stefan M. Golas, Wilson Ho, Connor W. Coley, and Kevin M. Esvelt. PyLabRobot: An open-source, hardware-agnostic interface for liquid-handling robots and accessories. *Device*, 1(4):100111, 2023. doi: 10.1016/j.device.2023.100111.
- [44] James M. Wilson. Gantt charts: A centenary appreciation. *European Journal of Operational Research*, 149(2):430–437, 2003. doi: 10.1016/S0377-2217(02)00769-5.
- [45] Guo Yu, Alan Blackwell, Sam Wilson, and Luis Anunciação. Effects of timing on users’ agency during mixed-initiative interaction. In *Proceedings of the 31st International BCS Human Computer Interaction Conference (HCI 2017)*, Electronic Workshops in Computing. BCS, 2017. doi: 10.14236/ewic/HCI2017.35.
- [46] Junyi Zhou, Luyao Ge, Xiaobo Li, Lulu Guo, Linjiang Chen, Jun Jiang, and Weiwei Shang. Multi-task scheduling of self-driving laboratories under scientific constraints. *Chemical Science*, 2026. doi: 10.1039/d6sc03892a. Advance article, published online 4 August 2026.
- [47] Xiaochi Zhou, Patrick Bulter, Changxuan Yang, Simon D. Rihm, Thitikarn Angkanaporn, Jethro Akroyd, Sebastian Mosbach, and Markus Kraft. Ontology-to-tools compilation for executable semantic constraint enforcement in LLM agents. arXiv:2602.03439, 2026.

Concept	Definition (Rhylthyme construct)	Examples
Program	The whole schedule: a named set of tracks plus its resources and metadata (programId , tracks , resourceConstraints)	Thanksgiving dinner; a Western blot; a wedding run-of-show; a 45-minute HIIT session
Track	One line of work whose steps run one after another; parallel work goes in separate tracks	The turkey; the gel; the ceremony; the lower-body superset
Step	A single timed activity within a track (stepId , duration , task)	Roast 3 h; incubate 1 h; procession 5 min; 12 reps
Dependency	A condition on when a step may start, within or across tracks (startTrigger : after a step ends or starts, at an offset from program start, on a manual signal, on abort, or all/any of several)	Gravy starts when the turkey comes out; blocking starts after transfer; music cue fires when the doors open; rest starts after the last set
Offset	A signed delay applied to a dependency; negative offsets start a step before its anchor ends	Potatoes in 20 min before the roast finishes; pre-warm the reader 10 min before the plate is ready
Buffer	Setup or cleanup time attached to a step, or an explicit gap between two steps (preBuffer , postBuffer , afterStepWithBuffer)	Preheat before baking; centrifuge spin-down; stage reset between acts; equipment changeover
Fixed / variable / indefinite duration	Exact time; bounded time the executor may end early; open-ended time the executor ends	Bake 25 min; sauté 3–5 min; incubate overnight until confluent; hold until the speaker finishes
Resource and capacity	A named task type with a limit on concurrent use (task , maxConcurrent)	One oven; two thermocyclers; one PA system; one squat rack
Actor and attention	People available to attend steps, by type and count, and the fraction of a person each resource needs (actorTypes , actorsRequired , qualifiedActorTypes)	A stove burner needs one cook; a microwave a quarter of one; only a licensed operator runs the cytometer
Environment	A reusable document declaring the resources and actors of a workplace, referenced by programs (environment)	Home kitchen with two cooks; small academic lab; theater; gym floor
Choice	A decision point whose options gate downstream steps (choice , choiceId)	Chicken or tofu; run the optional wash; rain plan; beginner or advanced set
Replicate and stagger	The same track or step repeated n times, in parallel, staggered by a delay, or serially (replicates , batch_size , stagger)	Three trays of cookies 10 min apart; twelve samples; three landings on one runway
Priority and flex	Ordering for planners when something must give, and permission for a step to stretch to fill a gap (priority , flex)	Drop the garnish before the main; let the rest interval fill the gap
Makespan, slack, critical chain	Total resolved time; how early a track finishes; the dependency chain that determines the end	Dinner is 4 h 40; the toast track has 6 min to spare; the turkey chain sets the finish
Manual gate and abort path	A step that waits for the executor’s signal; steps that run only if another is aborted (manual , onAbort)	“Start when guests are seated”; discard and re-plate if contaminated

Table 1: Concepts common to human-executed schedules and how Rhylthyme expresses them.

Axis	CPM / PERT / RCPSP	ATC flow mgmt	Robot task scheduling	Lab automation orchestrators	Workflow engines	Rhyllhyme
Executor	human, offline	controllers, airlines, live	robots and human-robot teams	instruments, robots, guided manual steps	compute nodes	a person, live
Author	analyst, PM	traffic manager	roboticist	automation engineer	bioinformatician	domain expert or LLM agent
Temporal semantics	precedence + durations	slots, windows, replanned	STN/STNU, temporospatial	synchronization, stations	data-dependency DAG	wall-clock triggers incl. manual, indefinite followable timeline; feasibility, conflicts
Objective	makespan, NPV	delay cost under capacity	feasible or near-optimal	throughput under scientific constraints	reproducible, portable output	maxConcurrent per task; actors (attention in the CLI runner) variable and indefinite, executor-ended
Resources	renewable	sector, airport capacity	robot capabilities	stations	CPU, memory, containers	one schema, environments per workplace native MCP
Uncertainty	PERT beta, stochastic	stochastic demand	contingent durations	process variance	retries	
Portability	tool-specific	domain-specific	domain-specific	SiLA, Autoprotocol	containers, standards	
Agent interface	none	none	none	emerging (vendor and community MCP servers)	emerging (vendor MCP servers)	

Table 2: Positioning Rhyllhyme against the traditions in Section 2. Cells describe the mainstream of each tradition, not every system in it.